CSE 493L Section 5 Worksheet

Comparison of pointer types.

Rust type	Description	What happens when it's dropped (goes out of scope)?	Similar in Java	Similar in C	Similar in C++
`T`	Owned value. `T` may be primitive, struct, or enum.	`T` is destructed.	N/A	Normal variable.	Normal variable.
`&T`	Immutable (shared) reference. Points to a `T` that someone else owns.	Nothing happens.	N/A	Not-null `const T*` pointer	Not-null `const T*` pointer
`&mut T`	Mutable (exclusive) reference. Points to a `T` that someone else owns.	Nothing happens.	N/A	Not-null `T*` pointer	Not-null `T*` pointer
`Box<T>`	Smart pointer that owns `T`. `T` is on the heap.	`T` is destructed.	N/A	Not-null `T*` returned by `malloc()`. Responsible for calling `free()`.	Not-null `unique_ptr<T>`
`Arc<T>`	Atomicly reference counted smart pointer. Like `&T`, except that ownership of `T` is shared between all `Arc<T>`.	Reference count is decremented. When it reaches zero, T is destructed.	All non-primitive variables.	N/A	Not-null `shared_ptr<T>`

This page is a simplified summary for section only. Please use the readme as the official source of truth when doing the assignment.

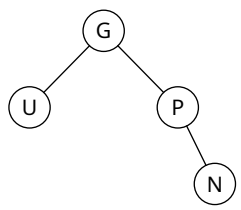
You may find this video helpful: https://youtu.be/A3JZinzkMpk

Red-black tree invariants

1. Every node is either red or black. The root node is black. Null nodes are considered black.
2. The children of a red node must both be black nodes.
3. For each node, all paths from that node to any leaf in its subtree must have the same number of black nodes, including the null nodes. This is called the node's *black height*.

Insertion node names

- N: newly inserted node.
- P: parent.
- G: grandparent.
- U: uncle.



All newly inserted nodes begin with the color red.

Simple insertion cases

1. Empty tree. The new node becomes the root and is colored to black.
2. There's already an existing node with this key. Just update its value.
3. The parent (P) of the newly inserted node is black. This means the insertion didn't violate any invariants, so we're done.

Complex insertion cases

If the parent (P) is red, we've violated invariant 2. How we fix this depends on the case:

1. The new node has a red uncle

Set P and U to black, and G to red. ("Push down the blackness of G".) This may introduce a new violation between G and its parent. Continue fixing them, treating G as a newly inserted node.

2. The new node has a black uncle

This will require different rotating and recoloring based on the arrangement:

`G`'s _____ child is `P`	`P`'s _____ child is `N`	Rotate	Swap colors
right	right	`G` left	`G` and `P`
right	left	`P` right, then `G` left	`G` and `N`
left	right	`P` left, then `G` right	`G` and `N`
left	left	`G` right	`G` and `P`

After you're done, if the tree root became red, change it back to black.

1.

```rust
fn increment_slice(slice: &mut [u32]) {
   for elem in slice {
       *elem += 1;
   }
}
fn main() {
   let mut list = Vec::from([1, 1, 1, 1]);
   increment_slice(&mut list[0..2]);
   assert_eq!(list, Vec::from([2, 2, 1, 1]));
}
```

Select all that apply:

❌Doesn't compile because `increment_slice()` gets a dangling reference.

The reference isn't dangling, because `list` is only dropped at the end of `main()`.

❌ Doesn't compile because `increment_slice()` doesn't know the length of slice.

A reference to a slice stores both its address and its length.

❌ Doesn't compile because `list` is moved into `increment_slice()`, so it can't be used again in `main()`.

`increment_slice()` is only given a reference to `list`, so `list` isn't moved.

❌ Doesn't compile because two mutable references to list exist at the same time.

A variable itself doesn't count as a reference. So the slice reference we pass to `increment_slice()` is the only reference.

❌ Compiles, but `increment_slice()` only affects its own copy of the slice, causing the assertion to fail.

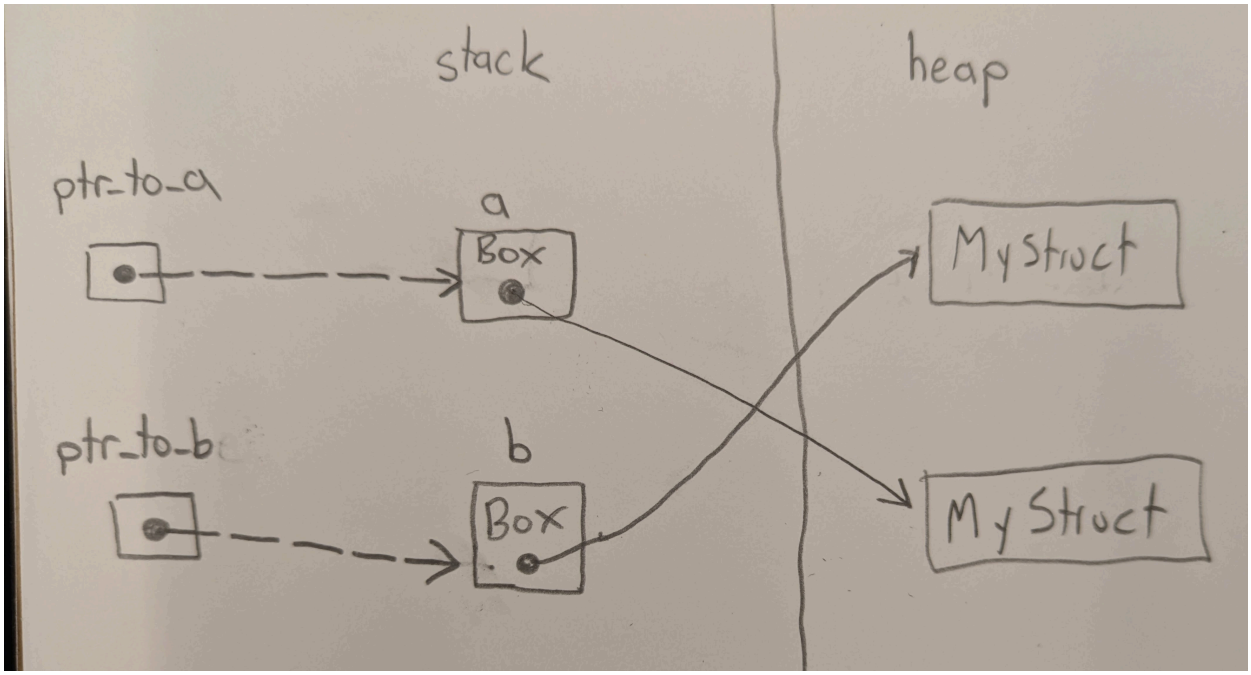
`increment_slice()` affects `list` because it's given a mutable reference to a slice of it.

✅ Compiles and the assertion succeeds.

2.

```rust
let mut a: Box<MyStruct> = Box::new(MyStruct::new());
let mut b: Box<MyStruct> = Box::new(MyStruct::new());
let ptr_to_a: &mut Box<MyStruct> = &mut a;
let ptr_to_b: &mut Box<MyStruct> = &mut b;

// Draw a memory diagram at this moment.
std::mem::swap(&mut a, &mut b);
// Draw another memory diagram at this moment.
println!("{a:?}, {b:?}");
```



We swap the contents of `a` and `b`. This means `a` now points to what `b` was pointing to, and vice versa.

3.

Show an example of an RBTree insertion which results in “pushing down color” resulting in another violation at the grandparent’s level. Draw an example of a valid tree with the new node immediately after it was inserted. Make sure to highlight the new node, then re-draw another re-colored tree. Point out where the new violation in the second tree occurs.

This is one of the questions for homework 5. We recommend reviewing the section 5 slides for inspiration.